2) To call the constructor of super class : [Constructor Chaining]

Whenever we write a class in java and we don't write any kind of constructor to the class then the java compiler will automatically add one default no argument constructor to the class.

THE FIRST LINE OF ANY CONSTRUCTOR IS RESERVED (OTHER FOR super() or this())super of Of this of that means first line of any constructor is used to call another constructor of either same class (this()) OR super class(super()).

It is the first line of any constructor if we don't specify either super() or that() then the java compiler will automatically add super() to the first line of constructor.

Now the purpose of this super() (added by java compiler), to call the default constructor or No-Argument constructor of the super class.

In order to call the constructor of super class as well as same class, we have total 4 cases.

Case 1 :

super() : It is **automatically** added by java compiler to the **first** line of the constructor, Used to call **default no argument OR no argument constructor of super class.**

//Program

```
class A
    {
        public A()
        {
            System.out.println("A class constructor");
        }
    }
class B extends A
    {
        public B()
        {
            System.out.println("B class constructor");
        }
    }
public class ConstructorChainingDemo1 {
    public static void main(String[] args)
    {
        B b1 = new B();
    }
}
```

* The following program explains default no argument constructor and super() is automatically added by java compiler.

```
package com.ravi.constructor_chaining;
class A
    {
        public A()
        {
            System.out.println("A class constructor");
        }
    class B extends A
    {
    }

    class C extends B
    {
        public C()
        {
            System.out.println("C class constructor");
        }
    }

    public class ConstructorChainingDemo2 {

        public static void main(String[] args)
        {
            C c1 = new C();
        }

    }
```

Case 2 :

super("RIT") : It must be written by developer in the first line of constructor to call parameterized constructor of **super class**. Here super class constructor will accept one parameter of type String.

```
package com.ravi.constructor_chaining;
class Super
    {
    public Super(String str)
    {
        super();
        System.out.println("Institute Name is :"+str);
    }
}
class Sub extends Super
{
    public Sub(String name)
    {
        super(name);
    }
}
public class ConstructorChainingDemo3
{
    public static void main(String[] args)
    {
        Sub s1 = new Sub("RIT");
    }
}
```

Case 3 :

this() : It must be written by developer to the first line of constructor.It is used to call no argument constructor of same class.

```
package com.ravi.constructor_chaining;
class Alpha
{
    public Alpha()
    {
        System.out.println("No Argument Constructor of Alpha class.");
    }
    public Alpha(int n)
    {
        this();
        System.out.println("Parameterized Constructor of Alpha Class :"+n);
    }
}
class Beta extends Alpha
{
    public Beta()
    {
        super(10);
        System.out.println("No Argument Constructor of Beta class.");
    }
}
public class ConstructorChainingDemo4 {
    public static void main(String[] args)
    {
        Beta b = new Beta();
    }
}
```

Case 4 :

this(10) : It must be written by developer to the first line of the constructor. It is used to call parameterized constructor of same class.

```
package com.ravi.constructor_chaining;

class Base
{
    public Base()
    {
        this(10);
        System.out.println("No Argument Constructor of Base class");
    }

    public Base(int i)
    {
        System.out.println("Parameterized Constructor of Base Class :"+i);
    }
}

class Derived extends Base
{
    public Derived()
    {
        System.out.println("No Argument Constructor of Derived class");
    }
}
public class ConstructorChainingDemo5
{

    public static void main(String[] args)
    {
        new Derived();
    }

}
```

The following program explains that we cannot execute non static member without executing super class constructor.

```
class Super{
    public Super(String str)
    {
        System.out.println(str);
    }
}

class Sub extends Super
{
    Sub()
    {
        super(getValue()); //error
    }
    public String getValue()
    {
        return "Hello";
    }
}

public class Test
{
    public static void main(String[] args)
    {
        new Sub();
    }
}
```
